StudyMate

STUDENT STUDY PLANNER — PROJECT REPORT

1. App Overview

Name: StudyMate — Student Study Planner. **Purpose:** A web app that helps students plan and track study activities. Users can add tasks with a subject, description and deadline; mark tasks as complete; edit or delete tasks; search and filter tasks; and view progress on a dashboard.

Target users: University and high-school students who need a lightweight planner that runs in a browser without requiring an account.

Repository: [Insert your GitHub repository link]

2. Design Decisions

I chose plain HTML, CSS and vanilla JavaScript because the assignment does not require a database. Browser localStorage is sufficient for a single-user planner, so adding a backend would introduce complexity without a clear benefit. Because the app has no build step, users and markers can run it by opening index.html directly. Implementing the logic manually also made state management explicit and easier to debug.

The main data entity is a Task { id, subject, description, deadline, completed }, with all tasks stored in one tasks array. This array is the single source of truth. The app saves it to localStorage under the key studymate_tasks after every change (saveTasks) and loads it at start-up (loadTasks). The task list, dashboard statistics, progress bar and upcoming-task section are all rendered from this array.

The interface has three views — Dashboard, My Tasks and Add Task — which are shown or hidden using CSS classes rather than separate pages. An editingTaskId variable allows one form to support both adding and editing: null indicates add mode, while an ID identifies the task being updated. Before user-entered text is inserted with innerHTML, it is escaped to prevent broken layouts or script injection.

3. Challenges & Solutions

Challenge 1 — Shared form state caused a bug

Using one form for both adding and editing required the form to track its current mode. When a user clicked “Add New Task” in the sidebar while editing, the form remained in edit mode: its fields stayed pre-filled and the button still read “Update Study Task”. A user intending to add a task could therefore overwrite an existing one. The navigation handler only reset the form when editingTaskId was null.

Solution: Reset the form every time the “Add New Task” view is opened from the sidebar. The Edit button now sets editingTaskId and populates the fields directly, bypassing that reset. I verified the fix by testing the sequence add → edit → cancel → add.

Challenge 2 — Storing data safely

localStorage writes can fail when storage is full or blocked, and stored values can become corrupted. Without error handling, a failed JSON.parse could stop the app from loading.

Solution: I wrapped loading and saving in try/catch blocks. If loading fails or the parsed value is not an array, the app starts with an empty task list. If saving fails, the user sees an alert explaining that the tasks could not be saved. I tested this by corrupting the stored value in browser DevTools and confirming that the app still starts.

Challenge 3 — Date shifting and invalid deadlines

JavaScript can interpret a date such as "2026-10-01" as UTC, which may display as the previous day in some time zones.

Solution: I split the YYYY-MM-DD value into its year, month and day components and construct a local date using new Date(year, month - 1, day) before formatting it. I also set the date input’s min value to today and re-check the deadline in JavaScript when the form is submitted, because the min attribute alone does not prevent every invalid input path. Users receive a clear message if they try to save a past deadline.

4. Testing & Reflection

I manually tested the app in [Chrome/Firefox/Edge] at desktop and mobile widths using the following checklist:

- Added a valid task; it appeared in the list and the dashboard counts updated.
- Submitted empty fields; validation feedback appeared and no task was created.
- Tried to set a past deadline; the app blocked the submission and displayed an error.
- Marked a task complete; its card showed “Undo”, and the counts and progress bar updated. Undo reversed the change.
- Edited a task; the form was pre-filled and the changes were saved.

- Deleted a task; a confirmation dialog appeared before removal and the dashboard updated.
- Searched by subject and description, including combining a search with a filter.
- Checked empty states: no tasks, no search results and no pending tasks.
- Refreshed the browser and confirmed that tasks persisted.
- Resized the page to approximately 375 px; the sidebar collapsed and task cards stacked.

Testing revealed two issues that I fixed: the edit-mode navigation bug described above, and overdue tasks being styled like ordinary pending tasks. Overdue tasks now have a distinct red badge. I also confirmed that corrupted localStorage data no longer prevents the app from starting.

Further improvements: With more time, I would replace alert() and confirm() with inline, non-blocking feedback such as toast notifications. I would also add due-time reminders and sorting by subject or status.

What I learned: Most of the work in a simple app lies in state management and edge cases, not just the happy path. Keeping one source of truth (the tasks array) and re-rendering the interface from it made features such as search, filtering and dashboard counts easier to implement. Shared form state and browser storage required careful handling. I also learned to test deliberately — including invalid input, corrupted storage and narrow screens — rather than testing only the workflow I expected users to follow.